Guía de aprendizaje · Semana 3: tablero interactivo con Shiny

Teoría de Probabilidades · Curso 013 · Proyecto de gemelo digital

Material de aprendizaje

2026

Contents

Propósitos de aprendizaje 1

Sesión 5 (90 min): interfaz e integración 1

Modelo conductor y convenciones 2

Sesión 6 (90 min): validación y preparación de sustentación 5

Actividad integradora 6

Producto de la semana y sustentación 6

Autoevaluación 6

> **Organización:** dos sesiones de 90 minutos. Equipos de máximo tres estudiantes. Ejemplo conductor: cafetería universitaria con **una fila y un servidor**. Los valores numéricos son **supuestos didácticos**, no mediciones del campus.

Propósitos de aprendizaje

Construir una interfaz Shiny que utilice un motor de simulación verificable, permita modificar parámetros y compare escenarios mediante indicadores y gráficos. Preparar una demostración breve y técnicamente fundamentada.

Requisito: tener instalado `shiny` (`install.packages("shiny")`, una sola vez). El ejemplo no necesita paquetes adicionales.

Sesión 5 (90 min): interfaz e integración

Tiempo	Actividad	Evidencia
0–20 min	Identificar entradas, salidas y eventos reactivos	Boceto de tablero

Tiempo	Actividad	Evidencia
20–45 min	Crear controles de parámetros y botón	Interfaz funcional
45–75 min	Conectar el motor y mostrar indicadores	Simulación ejecutable
75–90 min	Probar escenarios y errores	Lista de pruebas

Modelo conductor y convenciones

Durante las tres semanas se utilizará un mismo modelo para que los archivos sean independientes y los estudiantes puedan adaptar el código a parqueaderos, salas de estudio, cajas u otros sistemas.

Variable	Significado	Distribución supuesta	Parámetros
T	Tiempo entre llegadas (min)	Exponencial	tasa $\lambda = 0.3$ por minuto
S	Atención base (min)	Gamma	forma 4, escala 0.5
B	¿Compra adicional? (0/1)	Bernoulli	$p = 0.3$
A	Tiempo adicional si compra (min)	Uniforme	mínimo 0.5, máximo 1.5

El tiempo total de servicio es $D = S + BA$. El número de llegadas en una ventana de duración H **se deriva** de los tiempos entre llegadas; no se genera por separado para la misma trayectoria. Se supone independencia entre las variables primitivas, como simplificación que deberá revisarse con datos reales.

La intensidad de llegadas es $0.3 \times 60 = 18$ clientes por hora. La atención media es $E(D) = 4(0.5) + 0.3(1) = 2.3$ minutos. La utilización teórica aproximada de un servidor es $0.3(2.3) = 0.69$.

Ejemplo 1. Recuperar el motor de la semana 2

El siguiente código es autosuficiente: incorpora el motor y la aplicación completa. En el bloque siguiente se construye un archivo `app.R` con `writeLines`. El bloque **no se ejecuta automáticamente al tejer** el documento; ejecútalo manualmente en RStudio para crear la aplicación en la carpeta de trabajo.

```
app_codigo <- '
library(shiny)

simular_jornada <- function(lambda, forma=4, escala=0.5,
                            p_extra=0.3, horizonte=120) {
  stopifnot(lambda>0, forma>0, escala>0,
            p_extra>=0, p_extra<=1, horizonte>0)
  llegadas <- numeric(0)
  reloj <- 0
```

```
  repeat {
    reloj <- reloj + rexp(1, rate=lambda)
    if (reloj > horizonte) break
    llegadas <- c(llegadas, reloj)
  }
  n <- length(llegadas)
  if (n == 0) return(data.frame(llegada=numeric(0),
                                espera=numeric(0), servicio=numeric(0)))
  base <- rgamma(n, shape=forma, scale=escala)
  extra <- rbinom(n, size=1, prob=p_extra)
  servicio <- base + extra*runif(n, min=0.5, max=1.5)
  salida <- numeric(n)
  espera <- numeric(n)
  for (i in seq_len(n)) {
    libre <- if (i==1) 0 else salida[i-1]
    inicio <- max(llegadas[i], libre)
    espera[i] <- inicio-llegadas[i]
    salida[i] <- inicio+servicio[i]
  }
  data.frame(llegada=llegadas, espera=espera, servicio=servicio)
}

resumen_mc <- function(lambda, p_extra, B=200) {
  datos <- replicate(B, simular_jornada(lambda=lambda,
                        p_extra=p_extra), simplify=FALSE)
  n <- sum(vapply(datos,nrow,integer(1)))
  esperas <- unlist(lapply(datos, function(z) z$espera), use.names=FALSE)
  data.frame(clientes_medios=n/B,
             espera_media=if(n) mean(esperas) else NA_real_,
             prob_espera_5=if(n) mean(esperas>5) else NA_real_)
}

ui <- fluidPage(
  titlePanel("Gemelo probabilístico: cafetería"),
  sidebarLayout(
    sidebarPanel(
      sliderInput("lambda", "Llegadas esperadas por hora",
                  min=6, max=30, value=18, step=1),
      sliderInput("p_extra", "Probabilidad de compra adicional",
                  min=0, max=1, value=0.3, step=0.05),
      numericInput("B", "Réplicas Monte Carlo", value=200,
                   min=50, max=1000, step=50),
      actionButton("ejecutar", "Ejecutar simulación")
    ),
    mainPanel(
      h4("Resultados del escenario"),
      tableOutput("indicadores"),
```

```
      plotOutput("histograma"),
      h4("Comparación con escenario de referencia"),
      tableOutput("comparacion"),
      helpText("Referencia fija: 18 llegadas/hora y p=0.3.",
               "Modelo: un servidor, FIFO, horizonte 120 min.")
    )
  )
)

server <- function(input, output, session) {
  simulacion <- eventReactive(input$ejecutar, {
    set.seed(3100)
    actual <- resumen_mc(lambda=input$lambda/60,
                         p_extra=input$p_extra, B=input$B)
    set.seed(3100)
    referencia <- resumen_mc(lambda=18/60,
                             p_extra=0.3, B=input$B)
    set.seed(4100)
    trayectoria <- simular_jornada(lambda=input$lambda/60,
                                   p_extra=input$p_extra)
    list(actual=actual, referencia=referencia,
         trayectoria=trayectoria)
  }, ignoreNULL=FALSE)

  output$indicadores <- renderTable({
    round(simulacion()$actual, 3)
  }, rownames=FALSE)

  output$histograma <- renderPlot({
    z <- simulacion()$trayectoria$espera
    if (!length(z)) {
      plot.new(); text(0.5,0.5,"No hubo llegadas")
    } else {
      hist(z, breaks="FD", col="gray85", border="white",
           main="Tiempos de espera: una jornada",
           xlab="Minutos")
      abline(v=5, lty=2, lwd=2)
    }
  })

  output$comparacion <- renderTable({
    x <- rbind(Referencia=simulacion()$referencia,
               Actual=simulacion()$actual)
    round(x,3)
  }, rownames=TRUE)
}
```

```
shinyApp(ui, server)
'
writeLines(app_codigo, 'app.R')
shiny::runApp('app.R')
```

Explicación por componentes:

- `ui` define controles y espacios de salida. `sliderInput` modifica supuestos y `numericInput` controla el número de réplicas.
- `server` contiene la lógica reactiva. `eventReactive` **solo actualiza los resultados cuando se pulsa** «Ejecutar simulación»; cambiar un control no recalcula inmediatamente.
- `resumen_mc` agrega clientes de todas las réplicas, por lo que `prob_espera_5` es la proporción estimada entre **todos los clientes simulados**.
- La referencia usa 18 llegadas/hora y $p = 0.3$; el escenario actual toma los controles de la interfaz. Se usan semillas fijas para facilitar comparaciones reproducibles, sin afirmar que se eliminó todo el error Monte Carlo.
- El histograma corresponde a **una jornada** y la tabla a **muchas réplicas**; sus resultados no tienen por qué coincidir.

Ejemplo 2. Comprobación independiente antes de abrir Shiny

```
lambda_hora <- 18
p_extra <- 0.3
servicio_medio <- 4*0.5 + p_extra*(0.5+1.5)/2
utilizacion_aprox <- (lambda_hora/60)*servicio_medio
knitr::kable(data.frame(servicio_medio_min=servicio_medio,
                        utilizacion_aprox=utilizacion_aprox),digits=3)
```

servicio_medio_min	utilizacion_aprox
2.3	0.69

Explicación: la utilización aproximada debe permanecer por debajo de 1 para que el sistema con un servidor tenga estabilidad en el largo plazo. La simulación finita sigue siendo ejecutable incluso si el usuario elige parámetros que generan sobrecarga; el informe debe señalarlo.

Sesión 6 (90 min): validación y preparación de sustentación

Tiempo	Actividad	Evidencia
0–20 min	Verificar resultados y casos límite	Lista de pruebas

Tiempo	Actividad	Evidencia
20–45 min	Comparar escenario base y alternativo	Tabla de sensibilidad
45–70 min	Documentar supuestos y limitaciones	Informe final
70–90 min	Ensayar demostración de 8 minutos	Guion de sustentación

Ejemplo 3. Pruebas de sensibilidad recomendadas

Ejecutar la aplicación en estas configuraciones y registrar la dirección del cambio, sin exigir que cada ejecución arroje el mismo resultado:

Prueba	Llegadas/h	Probabilidad extra	Qué revisar
Base	18	0.30	Indicadores de referencia
Menor demanda	12	0.30	Espera típicamente menor
Mayor demanda	24	0.30	Espera típicamente mayor
Más compras	18	0.70	Mayor duración media del servicio

La variación aleatoria puede invertir diferencias pequeñas. Si una conclusión parece contradictoria, aumentar las réplicas y comprobar el algoritmo antes de interpretarla.

Actividad integradora

Adaptar el tablero al proyecto del grupo. Incluir controles para **dos parámetros relevantes**, un botón de ejecución, **tres indicadores**, al menos **un gráfico**, y una comparación de **dos escenarios**. Presentar evidencia de una prueba de consistencia, de un escenario extremo y de una limitación explícita. Si se dispone de observaciones reales, contrastar al menos un indicador con ellas; si no, identificar el modelo como **prototipo basado en supuestos**.

Producto de la semana y sustentación

Entregar el código de la aplicación (`app.R` o carpeta Shiny), el informe breve de validación y una presentación. Para la semana 4, preparar: problema (1 min), variables y parámetros (2 min), algoritmo (1 min), demostración (3 min), conclusiones (1 min), más preguntas. La guía oficial establece la presentación final y tablero para el **12 de noviembre**.

Autoevaluación

- ¿La aplicación responde al botón de simulación y modifica resultados?
- ¿El código permite reproducir los experimentos?
- ¿Podemos explicar cada distribución y cada indicador?
- ¿La comparación mantiene claros los parámetros de referencia?
- ¿Hemos documentado las limitaciones del prototipo?